

P3441R2

Rename `simd_split` to `simd_chunk`

Published Proposal, 2025-01-30

This version:

<http://wg21.link/P3441R2>

Authors:

[Daniel Towner](#) (Intel)

[Ruslan Arutyunyan](#) (Intel)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

The behaviour of `simd_split` is inconsistent with `ranges::views::split`, but is consistent with the behaviour of `ranges::views::chunk`. We propose to rename `simd_split` to `simd_chunk` to reflect this similarity in behaviour. We also propose that new overloads of `simd_chunk` which take an integer parameter are provided as a convenience for a common use case.

Table of Contents

- 1 Revision History
- 2 Motivation
- 3 Implementation experience
- 4 Wording
 - 4.1 Modify [`simd.syn`]
 - 4.2 Modify [`simd.creation`]

§ 1. Revision History

R1 => R2

- Improved examples
- Added more background to naming choice and motivation provide an integer overloads.
- Added implementation experience
- Improved wording to fix some errors and to provide more context for required changes.
- Added revision history

R0 => R1

- Renamed `simd_chunk_n` to `simd_chunk`.
- Wording changes to separate each overload to have its own clauses.

§ 2. Motivation

The `simd_split<T>` function takes a `basic_simd` object and breaks it down into a tuple of as many objects of type `T` that it can, and maybe one remainder object. An example illustrates a practical use of this, where a large `basic_simd` value is broken into many smaller pieces, each of which fits a specific hardware register type (possibly with a hardware-specific abi):

```
using Avx2RegisterType = simd<float, 8>; // Or with a hardware ABI instead.

simd<float, 19> x;
auto t = simd_split<Avx2RegisterType>(x);
// get<0>(t) will be of type simd<float, 8>
// get<1>(t) will be of type simd<float, 8>
// get<2>(t) will be of type simd<float, 3> - the remainder

// Note that each element is now the right size to pass to an AVX2 intrinsic.
```

If the original type is perfectly divisible into type `T` then an `array<T>` is returned instead of a tuple of different sized `basic_simd` objects.

The behaviour of `simd_split` is virtually identical to that of `ranges::views::chunk` and `ranges::chunk_view`. They take a view and a number `n` and produce a range of views (the chunks) of the original view, such that each chunk, except maybe the last one, has the size `n`.

In contrast, `ranges::views::split` has a different behaviour to the similarly named `simd_split`. The ranges version of `split` takes an input range and a delimiter value, and generates a range of views split on the delimiter. For example, the string "This,is,a,list", when split by the comma value would generate a range containing 4 views: "This", "is", "a", "list".

We propose that the `simd_split` function is renamed to `simd_chunk` to make its behavior consistent with the existing range/view counterparts. We did not consider any alternative names since they would introduce yet another name for a behavior that already exists and the intent is to allow the behavior of the existing term *chunk* to be reused in the context of `simd`.

A common use case for `simd_split/simd_chunk` is to break a larger `basic_simd` object into smaller native-sized pieces to call target-specific intrinsics, as illustrated in the first example in this paper. For that use-case the behaviour of `simd_chunk` is sufficient, but another common use case is where an algorithm requires that a `basic_simd` be broken down into pieces of a particular size. Only the size is of interest, not other details such as the ABI. With the current behavior the user would have to store the size inside a special type created just for that purpose, but it would be more convenient to provide overloads which take the size directly. The difference is illustrated here:

As existing	With new overloads
<pre>constexpr int ChunkSize = ...; simd<float, 19> x; // Create a simd type purely as a vehicle to // to pass around `ChunkSize`. using ChunkType = resize_simd_t<ChunkSize, simd<float, 19>>; auto t = simd_chunk<ChunkType>(x);</pre>	<pre>constexpr int ChunkSize = ...; simd<float, 19> x; // Use the ChunkSize directly. auto t = simd_chunk<ChunkSize>(x);</pre>

The overloaded version allows the user to directly use the size without having to convert to a chunk-type as a vehicle to convey the size indirectly. Allowing the size to be used directly allows the code to be simpler and more obvious.

§ 3. Implementation experience

The rename made no difference to the implementation, but does make the intent of those functions more obvious to those already familiar with the uses of the words "split" and "chunk" within the ranges libraries.

The extra overloads make places where chunking is performed for algorithmic reasons (rather than hardware-related type reasons) more obvious and readable. There is no need for the user's code to introduce new sized-types for chunking.

The implementation of the new overloads is trivial and was done for Intel internal implementation of `std::simd`. The overloads can do as the examples above illustrate, and convert the incoming type to a new size and call the existing `basic_simd` overloads.

§ 4. Wording

The wording diff is against the current C++ working draft.

§ 4.1. Modify [simd.syn]

Add new `simd_chunk` overloads immediately after the existing ones.

```
template<class T, class Abi, contiguous_iterator I, sized_sentinel_for<I> S, class... Fla
    requires indirectly_writable<I, T>
    constexpr void simd_partial_store(const basic_simd<T, Abi>& v, I first, S last,
        simd_flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, sized_sentinel_for<I> S, class... Fla
    requires indirectly_writable<I, T>
    constexpr void simd_partial_store(const basic_simd<T, Abi>& v, I first, S last,
        const typename basic_simd<T, Abi>::mask_type& mask, simd_flags<Flags...> f = {});

// [simd.creation], basic_simd and basic_simd_mask creation
template<class V, class Abi>
    constexpr auto
        simd_split simd_chunk(const basic_simd<typename V::value_type, Abi>& x) noexcept;
template<class M, class Abi>
    constexpr auto
        simd_split simd_chunk(const basic_simd_mask<mask_element_size<M>, Abi>& x) noexcept;

template<size_t N, class T, class Abi>
    constexpr auto
        simd_chunk(const basic_simd<T, Abi>& x) noexcept;
template<size_t N, size_t Bytes, class Abi>
    constexpr auto
        simd_chunk(const basic_simd_mask<Bytes, Abi>& x) noexcept;
```

§ 4.2. Modify [simd.creation]

```
template<class T, class Abi>
constexpr auto simd_split simd_chunk(const basic_simd<typename T::value_type, Abi>& x) noexcept
template<class T, class Abi>
constexpr auto simd_split simd_chunk(const basic_simd_mask<mask-element-size<T>, Abi>& x) noexc
```

Constraints:

- For the first overload T is an enabled specialization of basic_simd. If basic_simd<typename T::value_type, Abi>::size() % T::size() is not 0 then resize_simd_t<basic_simd<typename T::value_type, Abi>::size() % T::size(), T> is valid and denotes a type.
- For the second overload T is an enabled specialization of basic_simd_mask. If basic_simd_mask<mask-element-size<T>, Abi>::size() % T::size() is not 0 then resize_simd_t<basic_simd_mask<mask-element-size<T>, Abi>::size() % T::size(), T> is valid and denotes a type.

Let N be $x.size() / T::size()$.

Returns:

- If $x.size() \% T::size() == 0$ is true, an array<T, N > with the i^{th} basic_simd or basic_simd_mask element of the j^{th} array element initialized to the value of the element in x with index $i + j * T::size()$.
- Otherwise, a tuple of N objects of type T and one object of type resize_simd_t< $x.size() \% T::size()$, T>. The i^{th} basic_simd or basic_simd_mask element of the j^{th} tuple element of type T is initialized to the value of the element in x with index $i + j * T::size()$. The i^{th} basic_simd or basic_simd_mask element of the N^{th} tuple element is initialized to the value of the element in x with index $i + N * T::size()$.

```
template<size_t N, class T, class Abi>
constexpr auto simd_chunk(const basic_simd<T, Abi>& x) noexcept;
```

Effects: Equivalent to: return simd_chunk<resize_simd_t<N, basic_simd<T, Abi>>>(x);

```
template<size_t N, size_t Bytes, class Abi>
constexpr auto simd_chunk(const basic_simd_mask<Bytes, Abi>& x) noexcept;
```

Effects: Equivalent to: return simd_chunk<resize_simd_t<N, basic_simd_mask<Bytes, Abi>>>(x);